

· [KLDP.org](#) · [KLDP Wiki](#) · [KLDP.net](#) · [통합 검색](#) ·



[Subversion Book/Branching And Merging](#)

[🔍](#) [📄](#) [🔄](#) [🔍](#) [🔍](#) [🔍](#) [🔍](#) [🔍](#) [🔍](#) [XML](#)

[FrontPage](#) | [FindPage](#) | [TitleIndex](#) | [RecentChanges](#) |

[SubversionBookRemake](#) › [SubversionBook/Preface](#) › [SubversionBook/Introduction](#) › [SubversionBook/GuidedTour](#) › [SubversionBook/BasicConcepts](#) › [SubversionBook/BranchingAndMerging](#)

Login:

Password:

[Join](#)

You have an ability to sense and know higher truth.

[LFS/Building](#)
[CategoryQuickstart](#)
[조정래](#)
[Doxygen/강좌01](#)

1Chapter. 브랜치(branch)와 merge

Table of Contents

- 1.1.
- 1.1. [브랜치\(branch\)란?](#)
- 1.2. [브랜치\(branch\)의 이용](#)
 - 1.2.1. [브랜치\(branch\)의 작성](#)
 - 1.2.2. [자신용의 브랜치\(branch\)에서의 작업](#)
 - 1.2.3. [이 이야기의 교훈](#)
- 1.3. [브랜치\(branch\)를 또 있고로 변경을 카피하는 것](#)
 - 1.3.1. [특정의 변경점의 카피](#)
 - 1.3.2. [반복 merge 문제](#)
 - 1.3.3. [브랜치\(branch\) 전체의 merge](#)
- 1.4. [저장소\(repository\)로부터의 변경을 삭제한다](#)
- 1.5. [작업 카피의 변환](#)
- 1.6. [태그](#)
 - 1.6.1. [간단한 태그의 작성](#)
 - 1.6.2. [복잡한 태그의 작성](#)
- 1.7. [브랜치\(branch\)의 관리](#)
 - 1.7.1. [저장소\(repository\)의 레이아웃](#)
 - 1.7.2. [데이터의 수명](#)
- 1.8. [통계](#)
- 1.9. [중간에 약간 수정한 사람의 이야기:](#)



1.1.

브랜치(branch), 태그(tag), 머지(merge)는 대부분의 버전 컨트롤 시스템에서 공통적으로 사용하는 개념입니다. 만약 이런 개념들에 친숙하지 않다면, 이 장은 좋은 소개글이 될 것입니다. 만약 이미 익숙한 내용이라면, 이 장을 통해 Subversion이 이런 개념들을 구현하고 있는지 알려주는 흥미로운 장이 되길 바랍니다.

브랜치(branch)는 버전 관리의 가장 기본이 되는 동작입니다. 데이터를 Subversion으로 사용한다는 것은 곧 이 기능에 의존하게 된다는 것을 의미합니다. 이 장에서는 당신이 Subversion의 기본 개념을 벌써 이해하고 있음을 전제로 합니다(>).

1.1. 브랜치(branch)란?

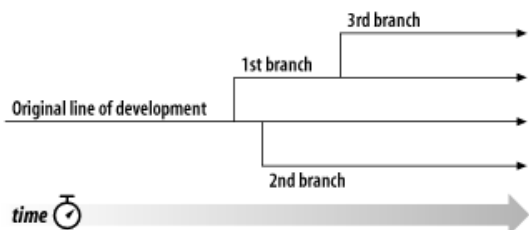
당신이 어떤 기업의 한 부서에서 문서(무언가에 대한 핸드북)의 관리를 맡고 있다고 생각해 봅시다. 어느 날 다른 부서로부터 당신이 관리하는 핸드북에서 어떤 부분이 조금만 바뀐 핸드북을 갖고 싶다는 요청을 했다고 합시다.

이때 당신은 어떻게 할까요? 대답은 뻔합니다. 문서의 복사본을 만들어 두 개의 복사본을 따로 관리할 것입니다. 각 부서에서 수정을 요구하면 각 부서에 해당하는 복사본을 수정하면 됩니다.

종종 양쪽 복사본 모두에서 똑같은 변경을 해야 할 경우도 있을 것입니다. 예를 들어 최초의 문서에 오타가 있을 경우 다른 복사본에도 분명 같은 실수가 있을 것입니다. 두 개의 문서는 거의 같으며 약간의 차이만 있을 뿐입니다.

이것이 *브랜치(branch)*의 기본 개념입니다. 즉 하나의 개발 흐름은 다른 흐름과 독립적으로 존재하지만 과거로 거슬러 올라가면 같은 조상을 공유하고 있는 것입니다. 브랜치(branch)는 항상 무엇인가로부터 복사되어 만들어진 뒤, 점점 원본과 멀어지면서 자신만의 역사를 만들어갑니다.

Figure 1-1. 개발의 브랜치(branch)



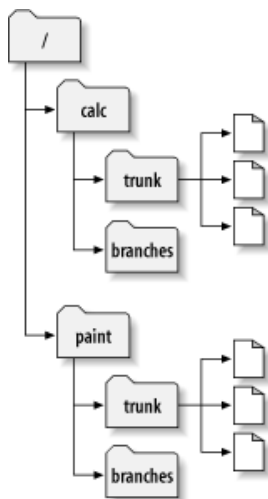
Subversion은 파일이나 디렉토리의 평행한 브랜치(branch)를 관리할 수 있습니다. 데이터를 카피해 새 브랜치(branch)를 만들거나 두 개의 버전이 어떤 관계를 가지고 있는지 기억할 수 있습니다. 어느 브랜치에 대한 수정을 다른 브랜치에도 적용할 수 있습니다. 마지막으로, 당신이 작업하고 있는 복사본의 일부분에다가 다른 브랜치들의 내용을 반영시킬 수도 있습니다. 즉, 당신은 매일매일 여러 개의 브랜치들을 서로 "혼합하고 붙일(mix and match)" 수 있습니다.

1.2. 브랜치(branch)의 이용

이 시점에서 당신은 각 커밋이 어떻게 저장소에서 완전히 새로운 파일시스템 트리("리비전(revision)"이라고 불리는)를 만드는지 알고 있어야 합니다. 혹시 모른다면, 뒤로 돌아가 리비전에 관한 >를 읽어 주세요.

이 장에서도 2장의 예를 사용합니다. 당신과 동료 Sally가 paint와 calc. 라는 두 개의 프로젝트가 있는 저장소(repository)를 공유하고 있었던 것을 떠올리세요. 그런데 이번에 누군가가 trunk와 branches, 이 두 개의 새로운 상부 디렉토리를 저장소(repository)에 추가했습니다. 이제 프로젝트는 이후에 설명할 trunk의 하위 디렉토리가 됩니다.

Figure 1-2. 저장소(repository) 레이아웃의 개시



이전과 같이 당신과 Sally 는 각각/trunk/calc 프로젝트 작업의 복사본을 가지고 있다고 합니다.

당신이 프로젝트의 재편성을 맡았다고 가정합시다. 그 작업에는 긴 시간이 필요하고 프로젝트의 모든 파일에 영향을 줍니다. 문제는 당신이 Sally에 간섭하고 싶지 않다는 것입니다. 그녀는 아직 여기저기에 있는 작은 버그를 잡고 있는 중이고, 이 작업을 하기 위해서 쉼터는 프로젝트의 최근 버전을 언제든지 사용할 수 있어야 합니다. 만약 당신이 자신의 변경을 조금씩 커밋하면 Sally의 작업은 확실히 중단되고 말 것입니다.

이런 문제들을 해결할 수 있는 방법이 있습니다. 당신과 Sally는 1, 2주간 정보를 공유하는 것을 그만둡니다. 즉, 자신의 작업 복사본에서 모든 파일에 대한 큰 작업을 시작하지만 그것이 완료할 때까지 커밋도 갱신도 하지 않는 방법입니다. 그러나 이것에는 여러가지 문제가 있습니다. 우선 안전하지 않습니다. 대부분의 사람은 작업 카피에 사고가 생길 것에 대비해서 틈틈이 저장소(repository)에 자신의 작업을 보존하는 것을 좋아합니다. 다음으로 이 전략은

유연성이 없습니다. 만약 여러분이 여러 대의 컴퓨터로 일을 진행하고 있다면 (아마 두 대 정도의 컴퓨터에 /calc/trunk 의 작업 복사가 있겠지요) 자신의 변경을 번거롭게 왔다갔다 하며 복사하면서 작업하거나 한 대의 컴퓨터에서만 작업을 해야만 합니다. 마지막으로 당신의 작업이 완료되었을 때 자신의 변경을 커밋하는 것이 몹시 어려운 것이라는 것을 알 수 있을 겁니다. Sally (또는 다른 멤버) 는 저장소(repository)에 각각 많은 다른 변경을 하고 있어 그것을 당신의 작업 카피에 merge하는 것은 큰 일이겠지요 작업을 단순히 끝내야 한다면 더욱 큰 문제가 됩니다.

좀 더 좋은 방식은 저장소(repository)에 자기만의 별도의 브랜치(branch), 즉 자기만의 작업라인을 만드는 것입니다. 이것은 다른 사람에게 간섭 받지 않고, 자신의 어중간한 작업을 가끔 보존할 수 있도록 합니다. 그러면서도 일부 정보에 대해서는 동료와 공유할 수 있습니다. 뒤에서 구체적으로 어떻게하는지를 설명 합니다.

1.2.1. 브랜치(branch)의 작성

브랜치(branch)의 작성은 매우 간단합니다 **svn copy** 커멘드를 이용해서 저장소(repository)에 있는 프로젝트를 복사하기만 하면 됩니다. Subversion는 하나의 파일만 복사할 수 있는 것이 아니라, 디렉토리 전체를 카피할 수도 있습니다. 당신이 /calc/trunk 디렉토리의 카피를 갖고 싶다고 가정 합니다. 새로운 카피는 어디에 두면 좋을까요? 어디에 두어도 상관이 없습니다. 어디에 두는지는 전적으로 프로젝트 팀의 정책에 따릅니다. 우리팀의 정책은 저장소(repository)의 /calc/branches 구역에 브랜치(branch)를 모아두는 것이고, 새로 만들 브랜치(branch) 이름은 "my-calc-branch" 로 합니다. /calc/trunk을 복사해서 /calc/branches/my-calc-branch라고 하는 새로운 디렉토리 를 만들 것입니다.

카피를 만드는 데에는 두 가지 방법이 있습니다. 개념을 확실히 하기 위해 복잡한 방법을 먼저 설명하겠습니다. 먼저 저장소(repository)의 루트 (/calc)를 작업 카피에 체크아웃 합니다:

```
$ svn checkout http://svn.example.com/repos/calc bigwc
A bigwc/trunk/
A bigwc/trunk/
A bigwc/trunk/Makefile
A bigwc/trunk/integer.c
A bigwc/trunk/button.c
A bigwc/branches
Checked out revision 340.
```

그리고는, **svn copy** 커멘드에 작업 카피 패스를 둘 건네주는 것만으로 카피를 만들 수 있습니다:

```
$ cd bigwc
$ svn copy trunk branches/my-calc-branch
$ svn status
A + branches/my-calc-branch
```

이 경우, **svn copy** 커멘드는 재귀적으로 trunk 작업 디렉토리의 내용을 새로운 작업 디렉토리 branches/my-calc-branch 에 카피합니다. **svn status** 커멘드로 확인할 수 있습니다만, 복사된 디렉토리는 저장소(repository)에 새로 더해질 것이라고 예약됩니다. 단, A의 뒤에, + 싸인이 표시되는데 주의해주세요. 이것은, 추가 예약이 완전히 새로운 것이 아니라, 무엇인가의 **카피** 인 것을 나타내고 있습니다. 변경을 커밋하면 Subversion은 네트워크 넘어로 작업 카피 데이터의 전체를 보내는 것이 것이 아니라, /calc/trunk를 카피하는 것으로 저장소(repository)에 /branches/calc/my-calc-branch 를 만듭니다:

```
$ svn commit -m "Creating a private branch of /calc/trunk. "
Adding      branches/my-calc-branch
Committed revision 341.
```

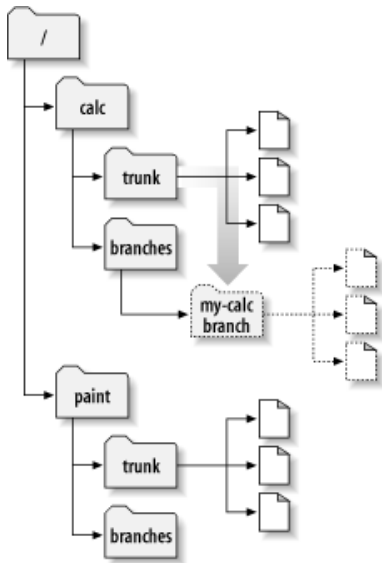
그런데, 브랜치(branch)를 만드는 좀 더 간단한 방법이 있습니다. 먼저 설명해야 마땅했습니다 하지만: **svn copy** 는 인수에 URL를 두 개 취하는 것이 할 수 있다고 하는 것입니다.

```
$ svn copy http://svn.example.com/repos/trunk/calc W
    http://svn.example.com/repos/branches/calc/my-calc-branch W
-m "Creating a private branch of /trunk/calc"
```

Committed revision 341.

이 두 개의 방법에는 아무 차이도 없습니다. 양쪽 모두 새로운 리비전 341의 디렉토리를 만들어, 새롭다 디렉토리는 /trunk/calc의 카피에 됩니다. 다만 두번째의 방법은 **동시에** 커밋도 발행합니다. [\[1\]](#) 두번째 방법(URL간 카피)은 즉시 커밋이 일어납니다. 이 방법은 저장소에 있는 많은 파일들을 체크 아웃 해오지 않기 때문에 보다 편리한 방법입니다. 실제로, 이 방법은 실제로는 카피가 전혀 일어나지 않습니다.

Figure 1-3. 새로운 카피가 있는 저장소(repository)



Cheap Copy

Subversion의 저장소(repository)는 특별한 설계를 가지고 있습니다. 디렉토리를 통째로 복사할 때, 저장소(repository)의 크기가 불필요하게 커지는 것을 걱정할 필요가 없습니다. Subversion은 실제로는 전혀 데이터를 카피하지 않습니다. 그 대신, 이미 원본 디렉토리에 링크되는 새로운 디렉토리를 생성합니다. UNIX의 예를 들면 이것은 hard-link의 개념과 비슷합니다. 이런 방식의 카피를 lazy copy 라고 부릅니다. 이 말은, 실제로 복사된 디렉토리에 커밋이 일어나기 전까지는 해당 파일의 내용이 변경되지 않는다는 것을 뜻합니다. 변경이 없는 나머지 파일들은 그대로 원본 디렉토리의 파일들에 링크되어 있게 됩니다.

이것이, Subversion 유저들이 "Cheap copy" 라는 말을 잘 듣게 되는 이유입니다. 카피가 일어날때 디렉토리의 크기에 대해 전혀 걱정할 필요가 없습니다. 카피에는 크기에 상관 없이 짧고 일정한 시간이 걸릴 뿐입니다. 이것이 Subversion에서의 커밋의 방식의 기본입니다. 각각의 리비전은 기존 리비전의 "Cheap copy"로, 그 중의 몇개 만의 아이템만이, 실제로 카피됩니다. (좀 더 알고 싶은 사람은 Subversion의 웹 사이트의, Subversion 설계 문서중의 "bubble up" 방식을 읽어 주세요)

당연히 카피와 데이터 공유의 메카니즘은 단순히 트리를 카피하고자 하는 사용자에게 숨겨집니다. 여기서의 핵심은 이런 방식이 카피는 시간과 공간에 대해 구애 받지 않는다는 점입니다. (the word 'cheap') 원하는 만큼 브랜치를 만들어서 사용하세요

1.2.2. 자신용의 브랜치(branch)에서의 작업

이것으로 프로젝트에 새로운 브랜치(branch)를 만들 수가 있었으므로, 새로운 작업 카피를 체크아웃 할 수 있습니다:

```
$ svn checkout http://svn.example.com/repos/branches/calc/my-calc-branch
```

```
A my-calc-branch/Makefile
A my-calc-branch/integer.c
A my-calc-branch/button.c
Checked out revision 341.
```

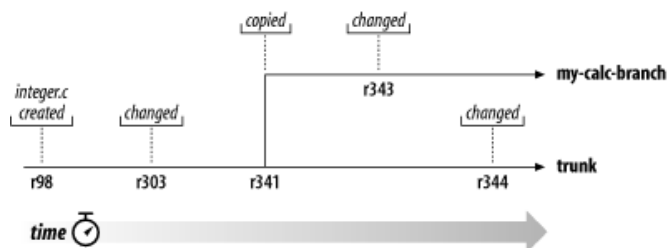
There's nothing special about this working copy; it simply mirrors a different directory in the repository. When you commit changes, however, Sally won't ever see them when she updates. Her working copy is of /calc/trunk. (Be sure to read the section called “Switching a Working Copy” later in this chapter: the `svn switch` command is an alternate way of creating a working copy of a branch.) 이 작업 카피 (working copy)에는 굳이 특별한 것은 없습니다. 단지 해당 저장소(repository)의 다른 디렉토리로의 미러링일뿐입니다. 당신이 변경을 커밋할때에도 Sally가 update과는 상관없습니다. (When you commit, She won't ever see them when she updates.) 그녀의 작업 카피는 /calc/trunk 입니다. (이번 chapter이후에 [작업 카피의 변환] 섹션을 꼭 읽으세요 : `svn switch` 명령어가 브랜치의 작업카피를 새로 만드는 것에 대한 것입니다.

일주일간이 경과하는 동안에, 이하의 커밋이 일어났다고 합시다:

- /branches/calc/my-calc-branch/button.c, (으)로 변경을 더해 리비전 342를 만들었다.
- /branches/calc/my-calc-branch/integer.c, (으)로 변경을 더해 리비전 343을 만들었다.
- Sally는 /trunk/calc/integer.c에 수정을 더해 리비전 344를 만들었다.

이것으로, integer.c에 두 개의 독립했다 개발 라인이 생겼습니다:

Figure 1-4. 어느 파일의 히스토리의 브랜치(branch)화



integer.c의 카피에 일어난 변경 히스토리를 보면(자) 재미있는 것을 압니다:

```
$ pwd
/home/user/my-calc-branch
```

```
$ svn log integer.c
```

```
-----
r343 | user | 2002-11-07 15:27:56 -0600 (Thu, 07 Nov 2002) | 2 lines
```

```
* integer.c: frozzled the wazjub.
```

```
-----
r303 | sally | 2002-10-29 21:14:35 -0600 (Tue, 29 Oct 2002) | 2 lines
```

```
* integer.c: changed a docstring.
```

```
-----
r98 | sally | 2002-02-22 15:35:29 -0600 (Fri, 22 Feb 2002) | 2 lines
```

```
* integer.c: adding this file to the project.
-----
```

Subversion은 카피된 시점까지의 integer.c 의 히스토리를 더듬고 있는 것에 주의해 주세요. (브랜치(branch)는 리비전 341 그리고 만들어진 것을 생각해 내 주세요). 이번은 Sally가 자신의 파일 의 카피에 대해서 같은 커멘트를 실행했을 때의 모습을 보겠습니다:

```
$ pwd
/home/sally/calc

$ svn log integer.c
```

```
r344 | sally | 2002-11-07 15:27:56 -0600 (Thu, 07 Nov 2002) | 2 lines

* integer.c:  fix a bunch of spelling errors.
```

```
r303 | sally | 2002-10-29 21:14:35 -0600 (Tue, 29 Oct 2002) | 2 lines

* integer.c:  changed a docstring.
```

```
r98 | sally | 2002-02-22 15:35:29 -0600 (Fri, 22 Feb 2002) | 2 lines

* integer.c:  adding this file to the project.
```

Sally는 자신의 리비전 344의 변경을 볼 수가 있습니다만, 당신이 리비전 343에 한 변경은 볼 수가 없습니다. Subversion에서는, 이 두 커밋은 다른 저장소(repository)의 장소에 있는 다른 파일에 대해 일어납니다. 그러나, Subversion은, 두 개의 파일이 공통의 히스토리를 가지고 있는 것을 **나타내도 있습니다**. 리비전 341 그리고 일어난 브랜치(branch) 카피 전은 양자는 같은 파일을 사용하고 있었습니다. Sally와 당신이 어느쪽이나 리비전 303으로 98을 볼 수가 있는 것은 기아제입니다.

1.2.3. 이 이야기의 교훈

이 마디에서의 중요 사항은 둘입니다.

1. 다른 많은 버전 관리 시스템과는 달라 Subversion의 브랜치(branch)는 저장소(repository)의 **보통 파일 시스템 디렉토리**로서 존재합니다. 특별한 구조가 있는 것은 아닙니다.
2. Subversion은 내부적으로는 "브랜치(branch)"라는 개념을 가지지 않습니다 그것은 단순한 복사본입니다. 단순히 디렉토리를 복사한 결과가 "브랜치(branch)"인 것은, **여러분이** 그러한 의미로 생각하기 때문입니다. 여러분이 그 디렉토리를 다른 의미로 생각하고 다르게 취급할 수도 있지만, 어쨌든 Subversion에게는 복사 의해 작성된 보통 디렉토리일 뿐입니다.

1.3. 브랜치(branch)를 또 있고로 변경을 카피하는 것

그런데, 당신과 Sally는 프로젝트 우에노타이라행 한 브랜치(branch)로 작업하고 있습니다. 당신은 자신의 사적인 브랜치(branch)로 작업하고 있어, Sally 는 *trunk*, 혹은, 개발의 주계 위에서 작업하고 있으면(자) 합니다.

많은 공헌자가 있는 것 같은 프로젝트에서는, 대부분의 사람들은 trunk의 카피를 가지고 있는 것이 보통입니다. trunk 를 부수어 버릴지도 모르다 같은 긴 기간을 걸친 변경을 더할 필요가 있는 경우는 항상, 표준적인 수속 (으)로서는 우선 사적인 브랜치(branch)를 만들어, 모든 작업이 완료할 때까지 변경 점을 그 브랜치(branch)에 커밋합니다.

그러한 방식의 이점으로서, 두 명의 작업은 서로 간섭하지 않는 곳입니다. 결점은 두 명의 작업 내용은 곧바로 **몹시** 차이가 나고는 끝내는 것입니다. "틀어박혀"전략의 문제의 하나는 자신의 브랜치(branch)의 작업이 완료할 경우에 일어나는 것을 생각해 내 주세요. 무섭고 많은 충돌 없이, 당신의 변경을 trunk에 merge 하는 것은 거의 불가능하겠지요.

그 대신에, 작업중에, 당신과 Sally는 변경을 계속 공유하는 것이 좋다 그렇지. 어떠한 변경이 공유하는 가치가 있는지는 당신이 결정한다 일입니다. Subversion를 사용하면(자) 브랜치(branch)간의 선택적인 "카피"를 할 수 있습니다. 그리고 브랜치(branch)상에서의 작업이 완전하게 끝나면(자), 브랜치(branch)상(?)변경점의 전체를 trunk에 써 되돌릴 수가 있습니다.

1.3.1. 특성의 변경점의 카피

전의 마디로, 당신과 Sally는 별브랜치(branch)상에서 `integer.c` (으)로 변경을 더했다고 했습니다. 만약 리비전 344의 Sally의 로그 메시지 (을)를 보면, 무엇인가의 스펠 미스를 고친 것을 알 수 있을지도 모릅니다. 이 경우 틀림없고, 같은 파일의 당신의 카피도 역시 같은 스펠 미스가 있을 것입니다. 이 파일에 대한 향후의 당신의 수정은 스펠 미스가 있다 장소에 영향을 줄지도 모르지 않고, 자신의 브랜치(branch)를 언젠가 merge 할 경우에 (은)는 충돌이 일어나 버립니다. 그렇게 될 정도로라면, 너무 심하게 된다 **전에**, Sally의 수정을 지금 받는 편이 좋을 것입니다.

svn merge 커멘트를 사용할 때가 왔습니다. 이 커멘드는, **svn diff** 에 매우 가깝다 친척이라고 하는 것을 알 수 있습니다. (이 커멘드는 3장으로 설명했습니다). 양쪽 모두 저장소(repository)중의 두 오브젝트를 비교해, 그 차이를 조사할 수가 있습니다. 예를 들어 **svn diff** 에 Sally가 리비전 344로 한 변경점을 정확하게 표시할 수가 있습니다:

```
$ svn diff -r 343:344 http://svn.example.com/repos/trunk/calc
```

```
Index: integer.c
```

```
=====
--- integer.c      (revision 343)
+++ integer.c      (revision 344)
@@ -147, 7 +147, 7 @@
     case 6:  sprintf(info->operating_system, "HPFS (OS/2 or NT)"); break;
     case 7:  sprintf(info->operating_system, "Macintosh"); break;
     case 8:  sprintf(info->operating_system, "Z-System"); break;
-    case 9:  sprintf(info->operating_system, "CPM"); break;
+    case 9:  sprintf(info->operating_system, "CP/M"); break;
+    case 10: sprintf(info->operating_system, "TOPS-20"); break;
     case 11: sprintf(info->operating_system, "NTFS (Windows NT)"); break;
     case 12: sprintf(info->operating_system, "QDOS"); break;
@@ -164, 7 +164, 7 @@
     low = (unsigned short) read_byte(gzfile); /* read LSB */
     high = (unsigned short) read_byte(gzfile); /* read MSB */
     high = high >> 8; /* interpret MSB correctly */
-    total = low + high; /* add them togethe for correct total */
+    total = low + high; /* add them together for correct total */

     info->extra_header = (unsigned char *) my_malloc(total);
     fread(info->extra_header, total, 1, gzfile);
@@ -241, 7 +241, 7 @@
     Store the offset with ftell() ! */

     if ((info->data_offset = ftell(gzfile)) == -1) {
-    printf("error: ftell() returned -1. Wn");
+    printf("error: ftell() returned -1. Wn");
     exit(1);
     }

@@ -249, 7 +249, 7 @@
     printf("I believe start of compressed data is %uWn", info->data_offset);
     #endif

-    /* Set postion eight bytes from the end of the file. */
+    /* Set position eight bytes from the end of the file. */

     if (fseek(gzfile, -8, SEEK_END)) {
         printf("error: fseek() returned non-zeroWn");
     }

```

svn merge 도 거의 같습니다. 차분을 화면에 표시하는 대신에, 그것은 **로컬인 수정분**으로서 직접 당신의 작업 카피에 적용 합니다:

```
$ svn merge -r 343:344 http://svn.example.com/repos/trunk/calc
```

```
U integer.c
```

```
$ svn status
```

```
M integer.c
```

svn merge의 출력은, 당신용의 `integer.c`의 카피가 패치되었다 결과입니다. 이것으로 Sally의 변경이 포함되게 되었던 그것은 trunk로부터 당신의 사적인 브랜치(branch)의 작업 카피에 카피되어 로컬인 수정의 일부가 되었습니다. 이 수정을 재검토해, 올바르게 동작하는 것을 확인하는 것은 당신의 일입니다.

개의 시나리오로서 그렇게 잘은 가지 않고, `integer.c`가 충돌 상태가 될 수도 있습니다. 표준적인 방법을 사용해 충돌을 해소하는지(3장을 봐 주세요), 결국 **merge**가 나쁜 아이디어였다고 생각했을 때에는, 포기해 **svn revert**로 로컬의 변경을 취소하는 일도 할 수 있습니다.

그러나, **merge**된 변경을 확인해, **svn commit**(을)를 걸치는 것이 보통입니다. 이것으로, 변경은 자신의 저장소(repository) 브랜치(branch)에 **merge**되었습니다. 버전 관리의 말투에서는, 이러한 브랜치(branch)간의 수정점의 카피를, 보통 *porting*에 의한 변경과 말합니다.

어째서 패치를 사용하지 않는거야?

그렇게 생각할지도 모릅니다. 특히 당신이 Unix 유저라면 그렇겠지요. 어째서 일부러 **svn merge** 같은 것을 사용하는지? 어째서 단지 OS에 붙어 있는 **patch** 커멘드를 사용해 같은 것을 하지 않는 것인지? 예를 들어:

```
$ svn diff -r 343:344 http://svn.example.com/repos/trunk/calc > patchfile
$ patch -p0 patchfile
Patching file integer.c using Plan A...
Hunk #1 succeeded at 147.
Hunk #2 succeeded at 164.
Hunk #3 succeeded at 241.
Hunk #4 succeeded at 249.
done
```

이러한 특별한 경우라면, 말씀하시는 대로. 무슨 차이도 없습니다. 그러나 **svn merge**는 **patch**에서는 할 수 없는 특수한 기능이 있습니다. **patch**로 사용할 수 있는 파일 형식은 매우 한정되고 있습니다. 그것은 단순히 파일 내용을 조금 변경할 수가 있을 뿐입니다. 복수의 파일이나 디렉토리의 추가, 삭제, 명칭 변경과 같은 **트리**를 변경하는 구조를 갖고 있지 않습니다. Sally의 변경이 새로운 디렉토리를 추가하는 것 같은 것이었다 경우, **svn diff**는 그것에 전혀 주의를 베풀 수 없을 것입니다. **svn diff**는 한정되었다 패치 형식의 출력을 하는 것만으로 간단하게는 표현할 수 없는 것이 있습니다. [\[2\]](#) 그러나 **svn merge** 커멘드는 작업 카피에 직접 일하는 것으로 트리의 변경을 표현할 수가 있습니다.

주의: **svn diff**와 **svn merge**는 매우 잘 닮은 컨셉을 갖고 있습니다만, 여러가지 경우로 다른 구문이 됩니다. 관련한 8장을 잘 읽는지, **svn help**(을)를 사용해 주세요. 예를 들어 **svn merge**는 작업 카피 패스를 인수로 합니다. 즉 트리의 변경을 적용하는 장소의 지정이 필요(이)가 됩니다. 이 지정이 없으면, 자주(잘) 이용되는 이하의 조작의 어느 쪽인지(을)를 실행하려고 하고 있다고 보입니다:

1. 현재의 작업 디렉토리에, 디렉토리의 변경점을 **merge**하려고 하고 있다.
2. 현재의 작업 디렉토리에 있는 같은 이름의 파일에 대해서, 어느 특정의 파일에 일어난 수정을 **merge**하려고 하고 있다.

디렉토리를 **merge**하려고 하고 있는 경우로, 목적의 패스를 지정하지 않았다 경우, **svn merge**는, 위에 올린 제일의 경우이라고 간주, 현재의 디렉토리의 파일에 대해서 적용하려고 합니다. 만약, 파일을 **merge**하려고 하고 있는 경우로, 그 파일(또는 같은 이름의 파일)이 작업 카피 디렉토리에 존재하고 있는 경우, **svn merge**는 제2의 경우이라고 간주, 같은 이름의 로컬 파일에 대해서 변경을 적용하려고 합니다.

상기 이외의 장소에 적용하고 싶은 경우에는 그것을 명시적으로 지정할 필요가 있습니다:


```
$ svn merge -r 343:344 http://svn.example.com/repos/trunk/calc my-calc-branch
U   my-calc-branch/integer.c
```

1.3.2. 반복 merge 문제

변경의 merge는 매우 단순한 일로 생각됩니다만 실제로는 귀찮은 물건입니다. 문제는, 만약 하나의 브랜치(branch)를 다른 브랜치(branch)에 대해서 변경점을 반복해 merge 하면(자), 잘못해 같은 변경을 **두 번**해 버릴지도 모르다고 하는 것입니다. 이런 것이 일어나도, 문제가 일어나지 않는 것도 있습니다. 파일을 패치 할 때, Subversion 는 파일이 벌써 변경되고 있 다 경우에는 거기에 깨달아, 아무것도 하지 않습니다. 그러나, 벌써 존재하고 있다 변경이 하 등의 방법으로 수정되고 있었을 경우, 충돌이 일어납니다. 이상적이게는 Subversion은 자동 적으로 한 번 이상의 변경의 적용을 피해야 합니다.

이것은 CVS와 Subversion를 포함하는, 많은 버전 컨트롤 시스템을 괴롭힐 수 있고 있는 문 제입니다. 현재, 이 문제를 Subversion로 막는 방법은 어느 변경을 merge 했는지를 주의해 기억해 두는 것입니다만, Subversion은 아직 그러한 기능 (이)가 없습니다. 브랜치(branch)의 디렉토리를 작성할 경우에는, 어느 리비전이 그 중에 만들어졌는지를 기억해 둘 필요가 있습니다 어디엔가 자신을 위해서(때문에) 삼가해 두는 것입니다. 리비전(또는 리비전의 범 위)를 작업 카피에 merge 할 때 그것도 함께 기억해 둘 필요가 있습니다. 만약 이 손의 정보 를 잊어 버렸을 경우는, `svn log -v branch-dir`의 출력을 조사하는 것에 의해 한번 더 찾아내 는 것이 할 수 있습니다. 다만, 중요한 (일)것은 각각의 merge는 사람의 손으로 주의해 주어, 이전에 merge 한 리비전을 한번 더 merge 하지 않게 하는 것입니다.

물론, Subversion은 이 문제를 릴리스 1.0이후의 어디선가 해결 할 계획이 있습니다. 이 merge에 관한 모든 정보는 속성의 메타데이터에 기록할 수가 있으므로, (>) 언젠의 날이나 Subversion은 자동적으로 반복해 merge를 자동적으로 피하는 것이 할 수 있게 되겠지요.

1.3.3. 브랜치(branch) 전체의 merge

지금, 생각해 온 예를 완결시키기 (위해)때문에, 조금 시간이 경과했다고 합니다. 며칠인가 경과해, 많은 변경이 trunk에도 선반의 사적인 브랜치(branch)에도 오코시. 그리고 당신은 사적인 브랜치(branch)상에서의 작업을 끝냈다고 합시다; 기능 추가, 또는 바그픽크스가 완 료해, 다른 사람이 그 부분을 사용할 수 있도록(듯이) 하기 위해서, 당신의 브랜치(branch)상 의 변경점의 모든 것을 trunk 에 merge 하고 싶다고 합니다.

그런데, 이러한 상황에서는, 어떻게 해 **svn merge** (을)를 사용하면 좋은 것일까요? 이 커멘 드는 두 개의 트리를 비교해, 그 차분을 작업 카피에 적용하는 것인 것을 생각해 내 주세요. 변경점 (을)를 받기 위해서(때문에)는, 당신은 trunk의 작업 카피를 손에 넣을 필요가 있습니 다. 여기에서는 당신은(완전하게 갱신된) 원래의 작업 카피를 아직 가지고 있는지, /trunk/calc의 새로운 작업 카피를 체크아웃 한 것 (와)과 가정합니다.

그러나, 어느 트리와 어느 트리를 비교하면 좋은 것일까요? 조금 생각하면(자), 그 대답은 분 명하게 생각됩니다: 단지 trunk의 최신의 트리와 당신의 브랜치(branch)의 최신의 트리입니 다. 그러나, 조심해 주세요 이 가정은 **잘못**해입니다. 그리고 이전 차이에, 대부분의 초심자는 당해 버립니다! **svn merge**는 **svn diff**와 같이 일하므로 마지막 트렁크와 브랜치(branch)의 트리의 비교는 단지 당신이 자신의 트리에 대해서 간 변경점만을 나타내는 것이 **아니다** 의를 알 수 있습니다. 그러한 비교는, 매우 많은 변경을 표시하겠지요: 그것은, 당신의 브랜치 (branch)에 대한 추가점만을 표시하는 것이 아니라, 당신의 브랜치(branch)에서는 결코 일 어나지 않았, trunk상의 변경점의 **취소**도 표시해 버리겠지요.

당신의 브랜치(branch)상에 일어난 변경만을 나타내려면 , 당신의 브랜치(branch)의 초기 상태와 최종적인 상태를 비교할 필요가 있습니다. **svn log** 커멘드를 당신의 브랜치(branch) 상에서 사용하면, 그 브랜치(branch)는 리비전 341으로 만들어진 것을 알 수 있습니다. 그리 고, 브랜치(branch)의 최종적인 상태는, 단지, HEAD 리비전을 지정하면 압니다.

결국, 최종적인 merge 처리는 이하와 같이 됩니다:

```
$ cd trunk/calc

$ svn merge -r 341:HEAD http://svn.example.com/repos/branches/calc/my-calc-branch
U   integer.c
U   button.c
```

```
U   Makefile
```

```
$ svn status
M   integer.c
M   button.c
M   Makefile
```

[examine the diffs, compile, test, etc.]

```
$ svn commit -m "Merged all my-calc-branch changes into the trunk. "
```

1.4. 저장소(repository)로부터의 변경을 삭제한다

svn merge 가 좋게 있는 사용법은 벌써 커밋했다 변경을 롤백(rollback) 하는 것입니다. `integer.c` (을)를 변경한 리비전 303의 변경은 완전하게 실수였다고 합니다. 그것은 커밋해야 하지는 않았습니다. 작업 카피의 변경을 취소하는데 (은)는 **svn merge** 를 사용할 수가 있습니다. 그 후로 로컬 의 변경을 저장소(repository)에 커밋합니다. 하지 않으면 안 되는 것은, **반대 방향**의 차분을 지정하는 것 뿐입니다:

```
$ svn merge -r 303:302 http://svn.example.com/trunk/calc
U   integer.c
```

```
$ svn status
M   integer.c
```

```
$ svn commit -m "Undoing change committed in r303. "
Sending      integer.c
Transmitting file data .
Committed revision 350.
```

저장소(repository) 리비전에 대한 하나 더의 생각은, 그것을 특정의 변경의 모임이라고 생각하는 것입니다(몇개의 버전 관리 시스템에서는, 이것을 *changesets*와 부르고 있습니다). `-r` 스위치를 사용해 **svn merge** 를 호출하는 것으로, 어느 체인지 세트를 적용하는지, 혹은 있는 범위의 체인지 세트 전부를 작업 카피에 적용할 수가 있습니다. 우리의 경우라면 **svn merge** (을)를 사용해 체인지 세트#303를 작업 카피에 **반대 방향으로** 적용합니다.

이러한 변경의 취소는, 보통 **svn merge** 의 조작에 지나지 않기 때문에, 작업 카피가 바라는 상태가 되었는지 여쭙는지는 **svn status** 와 **svn diff** 를 사용할 수가 있어 그 후 **svn commit** 로 저장소(repository)에 최종적인 버전을 보낼 수가 있다, 라고 하는 것을 눌러 두어 주세요. 커밋 후는 이 특별한 체인지 세트는 이미 HEAD 리비전에는 반영되지 않습니다.

이렇게 생각할지도 모릅니다: 로 하면(자), 그것은 「취소」가 아니다 (이)가 아닌가. 변경은 아직 리비전 303에 존재하고 있는 것은, 이라고. 만약 누군가가 `calc` 프로젝트의 리비전 303 (와)과 349의 사이의 버전을 체크아웃 했다고 하면(자), 잘못했다 변경을 받아들이는 것은 아닌지, 다른지, 라고.

말씀하시는 대로. 우리가, 변경의 것"취소"에 대해 말할 때, 사실은"HEAD로부터 없애는 것"을 말합니다. 원래의 변경은 저장소(repository)의 히스토리에 여전히 남아 있습니다. 대부분의 상황에서는 이것으로 충분합니다. 어쨌든 대부분의 사람들은 프로젝트의 HEAD를 뒤쫓는다 일에만 흥미가 있기 때문입니다. 그러나, 커밋에 관한 모든 정보를 삭제하고 싶다고 하는 예외적인 상황도 있겠지요. (아마, 누군가가 극비의 문서를 커밋해 버린, 등) 이것은 그렇게 쉬운 일이 아닙니다. Subversion은 의도적으로 결코 정보가 없어지지 않게 설계되고 있기 때문입니다. 히스토리로부터의 리비전의 삭제는, 연쇄적인 영향을 주어 모든 후속 리비전과 아마 모든 작업 카피에 혼란을 일으킵니다. [\[3\]](#)

1.5. 작업 카피의 변환

svn switch 커멘드는 존재하고 있는 작업 카피를 다른 브랜치(branch)로 변환합니다. 이 커멘드는 브랜치(branch)로 작업하는데 항상 필요라고 하는 것은 아닙니다만, 유저에 대해서 편리한 쇼트 컷 (을)를 준비합니다. 전의 예로, 사적인 브랜치(branch)를 만든 뒤, 그 새롭다 저장소(repository) 디렉토리의 작업 카피를 체크아웃 했습니다. 그렇게 한다 대신에, 단지

/trunk/calc 의 작업 카피를 새롭다 브랜치(branch)의 장소의 카피로 변경할 수가 있습니다:

```
$ cd calc
```

```
$ svn info | grep URL
```

```
URL: http://svn.example.com/trunk/calc
```

```
$ svn switch http://svn.example.com/branches/calc/my-calc-branch
```

```
U    integer.c
```

```
U    button.c
```

```
U    Makefile
```

```
Updated to revision 341.
```

```
$ svn info | grep URL
```

```
URL: http://svn.example.com/branches/calc/my-calc-branch
```

브랜치(branch)에 "스위치"한 후에는, 작업 카피의 내용은 그 디렉토리 (을)를 새롭게 체크아웃 했을 경우와 완전히 같은 것이 됩니다. 그리고 보통 이 커멘드를 사용하는 편이 보다 효율적입니다. 그렇다고 하는 것은, 대부분 브랜치(branch)는 아주 조금 내용이 다를 뿐입니다. 서버는 그 브랜치(branch) 디렉토리 (을)를 반영시키기 위해서(때문에) 작업 카피로 하지 않으면 안 되는 최소한의 변경만을 송신하면 됩니다.

svn switch 는 *--revision* (*-r*) 옵션을 취할 수도 있으므로, 항상 작업 카피 (을)를 브랜치(branch)의 "최신 상태"에로 옮길 필요가 있는 것은 아닙니다.

물론, 대부분의 프로젝트는 calc보다는 좀 더 복잡해, 복수의 서브 디렉토리를 포함하고 있습니다. Subversion 유저는 브랜치(branch)를 이용할 때에는 자주(잘), 특정의 방식을 합니다. :

1. 프로젝트의 '간(trunk)' 전체를 새로운 브랜치(branch) 디렉토리 에 카피한다.
2. '간(trunk)'의 작업 카피의 일부만을 브랜치(branch)에 밀러 한다.

바꾸어 말하면(자), 유저가 특정의 서브 디렉토리상에서만 브랜치(branch)의 작업이 일어난다 일을 알고 있는 경우에는 **svn switch**를 사용해 브랜치(branch)에 그 서브 디렉토리만을 이동합니다. (혹은, 쿨은 하나의 작업 파일 만을 브랜치(branch)에 switch 하는 것조차 있습니다!) 그방법에서는, 작업 카피 의 거의 모든 갱신을 보통 '간(trunk)'로부터 종래 대로 받는 것이 할 수 있습니다만, 바꾼 부분만큼은 변경되는 일 없이 남습니다(만약 브랜치(branch)에 대해서 누군가가 변경점을 커밋하기만 하지 않으면). 이 기능은 "혼합 작업 카피" 그렇다고 하는 개념에 완전히 새로운 차원을 덧붙이게 됩니다 작업 카피는 작업 리비전의 혼합을 포함하는 것이 가능한 한에서는 없고, 저장소(repository) 위치의 혼합도 포함할 수가 있습니다.

만약 작업 카피가 다른 저장소(repository) 위치로부터의 스위치 된 서브 트리 (을)를 몇개 인가 포함한다면, 그것은 계속 보통으로 기능합니다. 갱신하면(자), 각각의 서브 트리의 패치를 적절히 받겠지요. 커밋하면(자) 로컬 수정은 하나의 불가분의 변경을 저장소(repository)에 적용하겠지요.

저장소(repository) 위치의 혼합을 작업 카피에 반영시킬 수 있습니다만, 이러한 저장소(repository) 위치는 모두 **같은** 저장소(repository) 의 안에 없으면 안됩니다. Subversion의 저장소(repository)는 아직 서로 통신 할 수 없습니다. 이것은 Subversion 1.0 이후에 계획되고 있는 기능입니다. [\[4\]](#)

스위치와 갱신

svn switch 와 **svn update** 의 출력이 같은데 깨달았습니까? switch 커멘드는 실제로는 update 커멘드의 슈퍼 세트가 되어 있습니다.

svn update를 실행할 때, 그것은 저장소(repository)에 대해서 두 트리를 비교하도록(듯이) 요구합니다. 저장소(repository)는 그 비교를 실행 해, 클라이언트에 차분의 내용을 송신합니다. **svn switch** 와 **svn update** 의 유일한 차이는 update 커멘드는 항상 둘의 같은 패스를 비교한다고 한다 일입니다.

즉, 만약 작업 카피가 /trunk/calc의 카피라면 **svn update** 는 자동적으로/trunk/calc 의 작업 카피를 HEAD 리비전의/trunk/calc 와 비교 합니다. 만약 작업 카피를 브랜치(branch)로 옮기면(자), **svn switch** 는/trunk/calc 의 작업 카피를 무엇인가 HEAD 리비전의**다른** 브랜치(branch) 디렉토리 와 비교합니다.

바꾸어 말하면(자), 갱신은 시간의 방향으로 작업 카피를 움직입니다. switch 는 작업 카피를 시간**과** 공간의 양쪽 모두의 방향으로 작동시킵니다.

svn switch 는 본질적으로는**svn update** 의 변종이므로, 같은 동작을 공유합니다. 작업 카피 중의 어떠한 로컬의 변경도 저장소(repository)로부터 새로운 데이터가 닿을 때 보존됩니다. 이것으로 모든 영리한 잔기술이 (튼)물게 됩니다.

예를 들어/trunk 의 작업 카피가 있어 거기에 몇개인가 변경을 더했다고 합니다. 그리고 돌연, 사실은 브랜치(branch)에 하는 변경이었던 일로 눈치챱니다. 문제 없습니다. 작업 카피를**svn switch** 그리고 브랜치(branch)에 스위치 해도, 로컬의 변경은 그대로 남습니다. 그리고, 그것을 브랜치(branch)에 대해서 테스트해, 커밋할 수가 있습니다.

1.6. 태그

다른 버전 관리의 개념에, **태그**가 있습니다. 태그는 어떤 시점에서의 프로젝트의"snapshot"입니다. Subversion 그림 이 아이디어는 벌써 다양한 장소에 있는 것처럼 보입니다. 각각의 저장소(repository) 리비전은 확실히 그것입니다즉, 그것은 커밋 직후의 파일 시스템의 snapshot입니다.

그러나, 사람은 자주 태그에 대해서 인간에게 친숙함이 있는 이름을 붙이고 싶으면 생각하는 것입니다. 예를 들어,"release-1.0"과 같은. 또, 파일 시스템 의 좀 더 작은 서브 디렉토리의 snapshot를 갖고 싶은 일도 있습니다. 결국, 어느 소프트웨어의 일부의 release-1.0 이 리비전 4822의 특정의 서브 디렉토리 (이)라고 있는 것을 생각해 내는 것은 간단하지는 않습니다.

1.6.1. 간단한 태그의 작성

한번 더,**svn copy** 의 도움을 받습니다. 만약 HEAD 리비전의/trunk/calc 의 스냅 쇼트를 만들고 싶을 때에는, 그 카피를 취하면 좋기 때문에 했다:

```
$ svn copy http://svn.example.com/repos/trunk/calc W
    http://svn.example.com/repos/tags/calc/release-1.0 W
-m "Tagging the 1.0 release of the 'calc' project. "
```

Committed revision 351.

이 예에서는/tags/calc 디렉토리가 벌써 존재하고 있는 것으로 하고 있습니다. 카피 완료 후, 새롭다 release-1.0 디렉토리는, 당신이 카피했다 시점의 HEAD 리비전에 대해 프로젝트가 어떻게 보이고 있었는지를 스냅 쇼트로서 영원히 남기는 것입니다. 물론, 어느 리비전을 카피할까에 임해서 좀 더 정확하고 싶으면 생각할지도 모릅니다. 다른 사람이 당신이 보지 않을 때에 프로젝트에 대해 변경점을 커밋하고 있었을지도 모르지 않기 때문에. 만약 당신이 /trunk/calc 의 리비전 350이 자신의 가지고 싶은 스냅 쇼트라고 알고 있으면**svn copy** 커멘트에 **-r 350**을 지정할 수가 있습니다.

그렇지만 조금 기다려 주세요: 이 태그 작성의 수속은 브랜치(branch)를 만든다 위해(때문에) 사용해 온 수속과 같지 않아? 실은 그렇습니다. Subversion 에서는 태그와 브랜치(branch)에는 차이는 없습니다. 양쪽 모두 카피로 만들어진 보통 디렉토리입니다. 정확히 브랜치(branch)와 같이 카피된 디렉토리**가**"태그"이라고 말해지는 것은, 단지 **인간이** 그렇게 취급하기로 결정했기 때문에, 다만 그것 뿐입니다. 그 디렉토리에 아무도 커밋하지 않는 한, 그것은 영원히 snapshot로서 남습니다. 만약 누군가가 거기에 커밋 하기 시작하면(자), 그것은 브랜치(branch)가 됩니다.

만약 저장소(repository)를 관리하고 있다면, 태그를 관리하려면 2통의 방법이 있습니다. 최초의 어프로치는,"유저 맡김"입니다. 프로젝트 폴리시로서 당신의 태그를 두는 장소를 결정해 모든 유저에게 그 디렉토리를 카피할 때에는 어떻게 취급할까(을)를 알립니다. (즉, 모두가 거기에 커밋하지 않게 약속합니다) 두번째의 방식은 좀 더 가치가치입니다. Subversion 가 제공하는 액세스 제어 스크립트의 어떤 것인지를 사용해, 태그 area에는 새로운 카피를

만드는 것만이 할 수 있어, 그 이외의 조작을 금지합니다. (###cross-ref a section that demonstrates how to use these scripts?). 가치가치 방식은, 보통은 불필요합니다. 만약 유저가 잘못해 태그 디렉토리 (으)로 자신의 변경을 커밋해 버리면(자), 전의 장으로 설명한 방법으로 그 변경을 취소하면 좋기 때문에. 결국, Subversion은 버전 관리 시스템인 것입니다.

1.6.2. 복잡한 태그의 작성

가끔, 하나의 리비전의 하나의 디렉토리보다 좀 더 복잡한 snapshot를 갖고 싶은 일이 있습니다.

예를 들어, 프로젝트가 우리의calc 보다 좀 더 크다고 합니다. 많은 서브 디렉토리와 좀 더 많이 의 파일이 있다고 합니다. 일의 과정에서, 특정의 기능과 버그 수정을 포함했다 작업 카피가 필요하게 되었다고 판단합니다. 특정의 리비전의 파일과 디렉토리를 선택해(**svn update -r** liberally 를 사용해), 이것을 만들 수도 있고, 특정의 브랜치(branch)에 파일과 디렉토리를 스위치 하는 것에 의해도 할 수 있습니다. (**svn switch**를 사용한다) 이것을 하면 (자), 당신과 작업 카피는 다른 리비전으로부터 되는 다른 저장소(repository) 위치의 다음 벅기가 됩니다. 그러나 테스트 후, 자신이 확실히 필요와 하고 있는 편성인 것을 알 수 있었습니다.

자 snapshot를 취합니다. 하나의 URL를 작업하고 있지 않는 다른 장소에 카피합니다. 이 경우, 하고 싶은 것은 특정의 작업 카피 상태 그리고, 그것을 저장소(repository)에 격납하고 싶습니다. 행운의 일로 **svn copy** 는 실제로는 4 종류가 다른 사용법이 있습니다. (Chapter 8을 읽어 주세요) 그 중에는 작업 카피 트리 (을)를 저장소(repository)에 카피한다, 라고 하는 것도 있습니다:

```
$ ls
. /      ../      my-working-copy/

$ svn copy my-working-copy http://svn.example.com/repos/tags/calc/mytag

Committed revision 352.
```

이것으로 저장소(repository)에 새로운 디렉토리가 생겼습니다. /tags/calc/mytag입니다. 이것은 당신의 작업 카피 의 정확한 snapshot입니다 혼합 리비전, URL 그리고 방법이라고입니다.

다른 유저는 이 기능의 재미있는 사용법을 찾아냈습니다. 가끔, 자신의 작업 카피에 로컬인 수정을 했다 브랜치(branch)가 있지만, 그것을 다른 멤버에게 보여 주고 싶다고 하는 것 같은 상황입니다. **svn diff** 를 사용해 패치 파일을 보내는(그것은 트리의 변경을 취득할 수 없습니다) 대신에, **svn copy**를 사용해, 작업 카피를 저장소(repository)의 사적인 area에"업로드"합니다. 다른 멤버는 작업 카피를 새롭게 체크아웃 하는지,**cvs merge** (을)를 사용해 변경점만을 받을 수가 있습니다.

1.7. 브랜치(branch)의 관리

여기까지로, Subversion은 매우 유연한 시스템인 것이 알아 받을 수 있었는지라고 생각합니다. 디렉토리의 카피라고 하는 같은 기본적인 구조 위에 브랜치(branch)도 태그도 실장하고 있어, 브랜치(branch)도 태그도 보통 파일 시스템의 공간안에 있으므로, 많은 사람들은 Subversion의 구조에 놀랍니다. 그것은너무 유연한정도 입니다. 이 마디에서는 시간 경과와 함께 어떻게 데이터를 배치해 관리하는 것이 좋은 것처럼 붙여, 조금 설명합니다.

1.7.1. 저장소(repository)의 레이아웃

저장소(repository)의 편성에는 어느 정도, 표준화 된, 추천하는 방법이 있습니다. 대부분의 사람들은trunk디렉토리 에 개발의"주계", 브랜치(branch)의 카피가 있는branches 디렉토리, 그리고 태그의 카피가 있는tags디렉토리 (을)를 넣습니다. 저장소(repository)가 단 하나의 프로젝트를 포함한 경우에는 자주, 이 세 개의 디렉토리를 저장소(repository) 최상정도에 만듭니다:

```
/trunk
/branches
/tags
```

저장소(repository)가 복수 프로젝트를 포함한 경우는, 브랜치(branch)에 의해 배치를 인덱스화합니다. :

```
/trunk/paint
/trunk/calc
/branches/paint
/branches/calc
/tags/paint
/tags/calc
```

또 프로젝트에 따라서는 :

```
/paint/trunk
/paint/branches
/paint/tags
/calc/trunk
/calc/branches
/calc/tags
```

물론, 이 표준적인 레이아웃을 무시해도 괜찮습니다. 당신과 팀이 가장 작업하기 쉽게, 이 레이아웃은 어떻게 변화시켜도 괜찮습니다. 어떤 것을 선택해도 그것은 영구히 고정된 것이 아닙니다. 언제라도 저장소(repository)를 재편성 할 수가 있습니다. 브랜치(branch)와 태그는 보통 디렉토리 에 지나지 않기 때문에 **svn move** 커멘드를 사용하면 좋아하는 대로 이동, 명칭 변경을 할 수 있습니다. 어느 레이아웃으로부터 다른 레이아웃에의 변환은 단지 서버측에서의 몇회인가의 이동의 이야기가 됩니다. 만약 저장소(repository)중의 편성에 무엇인가 마음에 들지 않는 곳이 있다면, 디렉토리 에 관련한 잔기술을 사용해 주세요.

1.7.2. 데이터의 수명

Subversion 모델의 다른 좋은 기능으로서 다른 버전 관리된 아이템과 같이, 브랜치(branch)와 태그는 유한의 수명 밖에 가지지 않는다 같게도 할 수 있는 것입니다. 예를 들어 calc프로젝트 의 개인적인 브랜치(branch)상에서 모든 작업이 완료했다고 합니다. 모든 변경을, /trunc/calc에 merge 한 후에는 그 개인적인 브랜치(branch)의 디렉토리 완전히 소용 없게 됩니다:

```
$ svn delete http://svn.example.com/repos/branches/calc/my-calc-branch W
-m "Removing obsolete branch of calc project. "
```

Committed revision 375.

이것으로 브랜치(branch)는 없어져 버렸습니다. 물론 정말로 삭제되었다 (뜻)이유가 아닙니다: 디렉토리는 단지 HEAD 리비전으로부터 없어졌다 만으로, 아무도 번거롭게 할 수 있는 것은 없어졌습니다. 만약 이전의 리비전 (을)를 보고 싶은 경우는, (**svn checkout -r**, **svn switch -r**, 혹은 **svn list -r**를 사용해) 낡은 브랜치(branch)를 언제라도 볼 수가 있습니다.

삭제한 디렉토리를 열람하는 것 만으로는 불충분한 경우는, 언제라도 되돌릴 수도 있습니다. 데이터의 restore는 Subversion에서는 문제없습니다. HEAD에 되돌리고 싶은 삭제 디렉토리(또는 파일)가 있는 경우는, 단지 **svn copy -r** 를 사용해 낡은 리비전으로부터 카피해 주세요:

```
$ svn copy -r 374 http://svn.example.com/repos/branches/calc/my-calc-branch W
http://svn.example.com/repos/branches/calc/my-calc-branch
```

Committed revision 376.

우리의 예에서는 개인적인 브랜치(branch)는 비교적 짧은 생존 시간을 가집니다. 버그를 고치거나 새로운 기능을 추가하는데 이용했기 때문입니다. 작업이 끝나면, 브랜치(branch)의 수명도 거기서 마지막입니다. 그러나, 개발 내용에 따라서는 매우 긴 시간에 걸쳐서 두 개의 브랜치(branch)가 병행해 계속 사는 일도 있습니다. 예를 들어, 안정판의 calc 프로젝트를 공에 릴리스 할 때가 와, 그 소프트웨어의 버그를 취하는 데는 몇 개월 걸린다고 합니다. 릴리스 이전에 새로운 기능을 추가시키고 싶지는 않습니다만, 모든 멤버에게 개발을 중지하도록(듯이) 말해 구도 있어 선. 거기서 대신에, 당신은 그만큼 큰 수정은 발생하지 않는다 "안정판"의 브랜치(branch)를 만듭니다:

```
$ svn copy http://svn.example.com/repos/trunk/calc W
    http://svn.example.com/branches/calc/stable-1.0
    -m "Creating stable branch of calc project."
```

Committed revision 377.

이것으로 개발자는 최첨단의 기능(혹은 실험적인 기능)을 /trunk/calc에 계속 추가하는 것이 할 수 있어 버그 수정만을/branches/calc/stable-1.0에 커밋하는 것 같은 폴리시로 진행할 수가 있습니다. 즉, 멤버가 trunk상에서 계속 작업할 때, 버그 수정에 붙어 안정판 브랜치(branch)상에 가지고 갈 수가 있습니다. 안정판 브랜치(branch) 하지만 출시된 후에도, 그 브랜치(branch)를 긴 시간 들여 계속 메인테넌스한다 그렇지, 고객에 대해서 그 릴리스를 계속 서포트한다 한.

1.8. 통계

이 장에서는, 여러가지 기본에 접했습니다. 태그와 브랜치(branch)의 개념을 논의해, Subversion가 **svn copy**로 디렉토리를 카피한다 것에 의해 이러한 개념을 실장하고 있는 것을 설명했습니다. **svn merge**인 브랜치(branch)로부터 다른 브랜치(branch)로 변경점(을)를 카피하거나 잘못된 변경을 되돌리거나 하는 방법을 보여드렸습니다. **svn switch**를 사용해, 혼합 상태의 작업 카피를 만들어 보였습니다. 그리고, 어떻게 해 저장소(repository)내의 브랜치(branch)의 편성과 수명을 관리할까에 임해서 이야기했습니다.

Subversion에 대해, 이것만은 기억해 뒤 주세요: 브랜치(branch) 처리는 매우 가볍습니다. 좋아할 뿐(만큼) 사용해 주세요!

1.9. 중간에 약간 수정한 사람의 이야기:

이 글은 약간 실망스럽다 말한다 정도로 몹시 읽기 힘든 번역이다. 나는 갑자기 고치고 싶다는 생각이 갑작스레 들기 시작해 단지 약간 고쳐보았을 뿐이다. "가지 와 병합"에 심각한 부분만 골라내어 수정을 조금 가했다. 시간이 난다면 마저 고치고 싶다는 생각을(를) 하고 있다. 이 우스꽝스러운 section은 나중에 지우겠다.

Notes

[1] Subversion는 저장소(repository)간 카피를 서포트하고 있지 않습니다. **svn copy**나 **svn move**로 URL를 지정하는 경우, 같은 저장소(repository)내에서만 카피할 수가 있습니다.

[2] 장래적으로는 Subversion 프로젝트는 트리의 변경을 표현한다 같은 확장한 패치 형식을 사용할(있고는 개발할) 계획이 있습니다.

[3] 그렇지만, Subversion 프로젝트는 언제의 날인가 **svnadmin obliterate** 커멘트를 실장할 계획이 있습니다. 이것은 정보의 완전한 소거를 실행하는 커멘드입니다.

[4]

그러나, 서버상의 URL 가 변경되었지만, 기존의 작업 카피 (을)를 버리고 싶지 않은 경우에는, `--relocate` 스위치 돌출하고 **svn switch**를 사용할 수 있습니다. 보다 자세한 정보와 예에 대해서는

[EditText](#) | [Refresh](#) | [FindPage](#) | [DeletePage](#) | [LikePages](#) | [Tour](#) | [Keywords](#) |

Last modified: 2006-09-11 00:27:00, Processing time: 0.0400 sec

[일정관리프로그램](#)

Google에서 전사용 공유 캘린더를 무료로 제공. 지금 다운로드 하세요!

[업무용 웹하드 오피스 하드](#)

자체 구축형 웹하드 솔루션 판매 조직도 연동, 3중보안, 링크 검색